

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №16

Тема: Рекурсія. Схема роботи рекурсії. Форми рекурсивних процедур

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати поняття рекурсії, механізм роботи стека викликів при рекурсивному виконанні, класифікацію форм рекурсії (лінійна, деревоподібна, хвостова, взаємна), навчитися оцінювати ефективність рекурсивних алгоритмів і застосовувати мемоізацію для усунення надлишкових обчислень.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Визначення рекурсії. Рекурсія — це спосіб визначення функції (процедури, алгоритму), при якому вона викликає **сама себе**, безпосередньо або опосередковано, для розв'язання простіших підзадач того самого типу. Будь-яка коректна рекурсивна функція повинна містити:

- **Базовий випадок (base case)** — умову, при якій рекурсія припиняється і повертається результат без подальших викликів.
- **Рекурсивний випадок (recursive case)** — виклик функції самої себе з "меншими" (простішими, ближчими до базового випадку) параметрами.

2. Механізм стека викликів. Кожен виклик функції (зокрема рекурсивний) створює новий **кадр стека (stack frame)**, що містить локальні змінні, параметри та адресу повернення. При рекурсивному виклику новий кадр додається "поверх" попереднього; коли функція завершується (досягає базового випадку або оператора `return`), її кадр знімається зі стека, і виконання повертається до кадру, що викликав. Для `factorial(5)` стек послідовно накопичує кадри для $n=5, 4, 3, 2, 1$, а потім "розгортається" у зворотному порядку, обчислюючи добутки: $1, 2 \cdot 1=2, 3 \cdot 2=6, 4 \cdot 6=24, 5 \cdot 24=120$.

3. Форми рекурсії:

- **Лінійна рекурсія** — функція робить **один** рекурсивний виклик самої себе за виконання (наприклад, факторіал). Глибина стека пропорційна n .
- **Деревоподібна (множинна) рекурсія** — функція робить **кілька** рекурсивних викликів за виконання (наприклад, наївний розрахунок чисел Фібоначчі $\text{fib}(n)=\text{fib}(n-1)+\text{fib}(n-2)$, Ханойські вежі). Загальна кількість викликів може зростати експоненційно.
- **Хвостова рекурсія (tail recursion)** — рекурсивний виклик є **останньою** дією у функції, після якої більше немає обчислень над результатом виклику. Теоретично компілятор/інтерпретатор може оптимізувати хвостову рекурсію, перетворивши її на

цикл без додаткового росту стека (**оптимізація хвостових викликів**, TCO); проте стандартний інтерпретатор Python **не виконує** такої оптимізації, тому навіть хвостово-рекурсивні функції в Python витрачають стек пропорційно глибині рекурсії.

- **Пряма й непряма (взаємна) рекурсія.** Пряма рекурсія — функція викликає безпосередньо себе. Непряма (взаємна) рекурсія — функція А викликає функцію В, яка, своєю чергою, викликає функцію А (класичний приклад: перевірка парності/непарності числа через взаємні виклики `is_even/is_odd`).

4. Проблема надлишкових обчислень у деревоподібній рекурсії й мемоізація. Наївний рекурсивний розрахунок чисел Фібоначчі багаторазово обчислює одні й ті самі підзадачі (наприклад, `fib(3)` обчислюється кілька разів при обчисленні `fib(6)`), що призводить до експоненційної складності $O(2^n)$. **Мемоізація** — техніка динамічного програмування "згори вниз" (top-down), яка зберігає результати вже обчислених підзадач у допоміжній структурі (словнику, масиві) і повертає збережений результат замість повторного обчислення, знижуючи складність до лінійної $O(n)$.

5. Рекурсія проти ітерації. Будь-яку рекурсивну функцію можна переписати у вигляді ітеративного циклу (з явним використанням стека там, де це потрібно) — і навпаки. Рекурсія часто дає коротший і природніший запис для задач із рекурсивною структурою (обхід дерев, задачі типу "розділай і володарюй"), однак має накладні витрати на створення кадрів стека й обмежену максимальну глибину (у Python за замовчуванням ліміт становить 1000 викликів, хоча його можна змінити функцією `sys.setrecursionlimit`), що робить ітеративні розв'язки бажанішими для задач із дуже великою глибиною або жорсткими вимогами до продуктивності.

Хід роботи

Завдання 1. Ручна трасировка стека викликів для факторіала

Побудувати вручну діаграму стека викликів для обчислення `factorial(5)`, показавши як фазу "занурення" (накопичення викликів), так і фазу "розгортання" (повернення значень).

Порядок виконання:

1. Намалюйте (або опишіть у вигляді списку) послідовність викликів "занурення": `factorial(5)` викликає `factorial(4)`, який викликає `factorial(3)`, і так далі до `factorial(1)` — базового випадку.
2. Для кожного виклику вкажіть, який кадр стека додається (значення параметра `n`) і яка операція буде виконана після повернення з рекурсивного виклику (наприклад, "`factorial(5)` чекає на результат `factorial(4)`, щоб обчислити $5 * \text{result}$ ").
3. Зафіксуйте момент досягнення базового випадку (`n=1`, повертає 1) — це нижня точка "занурення", стек має максимальну глибину 5 кадрів.
4. Опишіть фазу "розгортання": як послідовно знімаються кадри зі стека (від `factorial(1)` до `factorial(5)`), і на кожному кроці обчисліть проміжний результат (1, потім $2 \cdot 1 = 2$, потім $3 \cdot 2 = 6$, потім $4 \cdot 6 = 24$, потім $5 \cdot 24 = 120$).
5. Намалюйте підсумкову схему у вигляді стовпчика кадрів стека (знизу — `factorial(1)`, зверху — `factorial(5)`) з позначенням, яке значення повертає кожен кадр.

Завдання 2. Ручна трасировка дерева викликів для наївного Фібоначчі

Побудувати вручну повне дерево рекурсивних викликів для обчислення `fib(6)` за формулою $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$, `fib(0)=0`, `fib(1)=1`, і порахувати загальну кількість викликів функції.

Порядок виконання:

1. Намалюйте кореневий вузол дерева "fib(6)", від якого відходять два дочірні вузли: "fib(5)" і "fib(4)".
2. Рекурсивно розкрийте кожен вузол "fib(k)" (де $k > 1$) на два дочірні вузли "fib(k-1)" і "fib(k-2)", доки не дійдете до листових вузлів "fib(1)" або "fib(0)" (базові випадки).
3. Зверніть особливу увагу на те, що деякі піддерева повторюються (наприклад, "fib(4)" з'являється в дереві кілька разів як окремий незалежний виклик) — обведіть або позначте кольором однакові за значенням аргумента піддерева, щоб наочно побачити дублювання обчислень.
4. Порахуйте загальну кількість вузлів (викликів функції) у побудованому дереві.
5. Порівняйте цю кількість з кількістю **унікальних** значень fib(k), які насправді потрібно обчислити (k від 0 до 6, тобто лише 7 унікальних підзадач) — зробіть висновок про масштаб надлишкових повторних обчислень.
6. Обчисліть кінцевий результат fib(6), підсумувавши значення від листків дерева до кореня, і перевірте: fib(6) має дорівнювати 8.

Завдання 3. Програмна реалізація та порівняння наївної й мемоїзованої версій Фібоначчі

Реалізувати мовою Python наївний рекурсивний, мемоїзований (з динамічним програмуванням) та ітеративний варіанти обчислення чисел Фібоначчі; порівняти кількість викликів функції для зростаючого n.

Порядок виконання:

1. Напишіть функцію `fib_naive(n, calls)`, що при кожному виклику збільшує лічильник `calls[0]` на 1, повертає n при $n \leq 1$, інакше рекурсивно повертає `fib_naive(n-1, calls) + fib_naive(n-2, calls)`.
2. Напишіть функцію `fib_memo(n, memo, calls)`, що спочатку перевіряє, чи значення n вже є у словнику `memo`: якщо так — одразу повертає збережений результат без подальшої рекурсії; інакше обчислює результат рекурсивно (аналогічно наївній версії), зберігає його в `memo[n]` перед поверненням.
3. Напишіть ітеративну функцію `fib_iterative(n)`, що обчислює числа Фібоначчі в циклі за допомогою двох змінних-акумуляторів без рекурсії взагалі.
4. Перевірте, що всі три реалізації дають однаковий результат для кількох значень n.
5. Для n від 5 до 30 (з кроком 5) виведіть кількість викликів функції для наївної та мемоїзованої версій — порівняйте темп їхнього зростання.

Приклад реалізації мовою Python:

```
python
def fib_naive(n, calls=None):
    if calls is not None:
        calls[0] += 1
    if n <= 1:
        return n
    return fib_naive(n-1, calls) + fib_naive(n-2, calls)

def fib_memo(n, memo=None, calls=None):
    if memo is None: memo = {}
    if calls is not None:
        calls[0] += 1
    if n in memo:
        return memo[n]
    if n <= 1:
```

```

    return n
    result = fib_memo(n-1, memo, calls) + fib_memo(n-2, memo, calls)
    memo[n] = result
    return result

def fib_iterative(n):
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(2, n+1):
        a, b = b, a+b
    return b

assert fib_naive(15) == fib_memo(15) == fib_iterative(15)
print("Коректність усіх реалізацій підтверджено\n")

print("Наївний рекурсивний Фібоначчі — кількість викликів функції:")
for n in [5, 10, 15, 20, 25, 30]:
    calls = [0]
    result = fib_naive(n, calls)
    print(f" fib({n}) = {result:>8}, викликів функції = {calls[0]:>10}")

print("\nМемоізований Фібоначчі — кількість викликів функції:")
for n in [5, 10, 15, 20, 25, 30, 50]:
    calls = [0]
    result = fib_memo(n, calls=calls)
    print(f" fib({n}) = {result:>15}, викликів функції = {calls[0]:>5}")

```

Результат виконання програми:

Коректність усіх реалізацій підтверджено

Наївний рекурсивний Фібоначчі — кількість викликів функції:

```

fib(5) =      5, викликів функції =      15
fib(10) =     55, викликів функції =     177
fib(15) =    610, викликів функції =    1973
fib(20) =   6765, викликів функції =   21891
fib(25) =  75025, викликів функції =  242785
fib(30) = 832040, викликів функції = 2692537

```

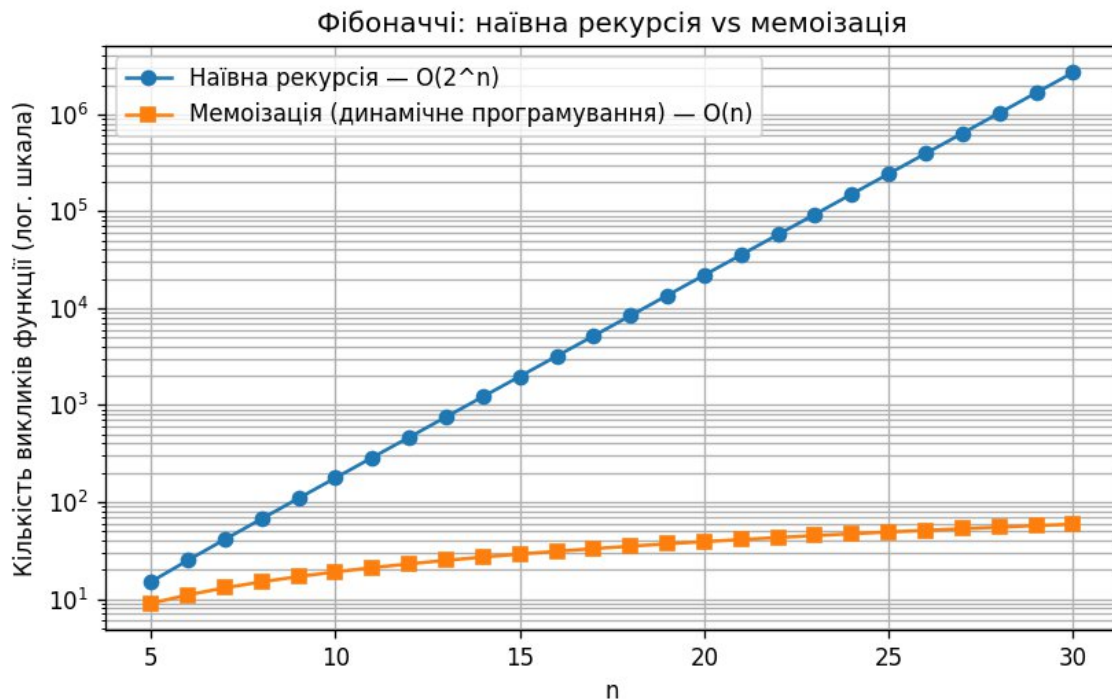
Мемоізований Фібоначчі — кількість викликів функції:

```

fib(5) =      5, викликів функції =      9
fib(10) =     55, викликів функції =     19
fib(15) =    610, викликів функції =     29
fib(20) =   6765, викликів функції =     39
fib(25) =  75025, викликів функції =     49
fib(30) =  832040, викликів функції =     59
fib(50) = 12586269025, викликів функції =    99

```

Графік порівняння:



Завдання 4. Реалізація Ханойських веж та дослідження хвостової рекурсії

Реалізувати класичну задачу Ханойських веж (деревоподібна рекурсія), перевірити теоретичну формулу кількості переміщень $2^n - 1$; реалізувати й порівняти хвостово-рекурсивний та звичайний варіанти обчислення факторіала.

Порядок виконання:

1. Напишіть рекурсивну функцію `hanoi(n, source, target, auxiliary, moves)`, що реалізує класичний алгоритм: щоб перемістити n дисків з `source` на `target` через `auxiliary`, спочатку рекурсивно перемістіть $n-1$ дисків з `source` на `auxiliary` (через `target`), потім перемістіть найбільший диск з `source` на `target` (один хід), потім рекурсивно перемістіть $n-1$ дисків з `auxiliary` на `target` (через `source`).
2. Кожне елементарне переміщення диска рахуйте, збільшуючи лічильник `moves[0]`.
3. Для n від 1 до 20 обчисліть фактичну кількість переміщень і порівняйте з теоретичною формулою $2^n - 1$.
4. Напишіть звичайну (не хвостову) рекурсивну функцію `factorial_recursive(n)`, де множення $n * \text{factorial_recursive}(n-1)$ виконується **після** повернення з рекурсивного виклику (тобто це НЕ хвостова рекурсія, оскільки над результатом виклику ще виконується операція множення).
5. Напишіть псевдо-хвостово-рекурсивну версію `factorial_tail(n, accumulator=1)`, де рекурсивний виклик `factorial_tail(n-1, n*accumulator)` є останньою дією функції (накопичувач передається як параметр, і множення виконується **до** виклику, а не після).
6. Поясніть у звіті (текстово, без коду), чому, попри хвостову форму `factorial_tail`, інтерпретатор Python усе одно витрачає $O(n)$ пам'яті стека для обох версій — оскільки

CPython не виконує оптимізацію хвостових викликів (ТСО), на відміну від деяких інших мов (наприклад, Scheme, Erlang).

- Порівняйте (експериментально, за допомогою `sys.getrecursionlimit()` та поступового збільшення `n`) максимальне значення `n`, для якого кожна з рекурсивних реалізацій факторіала й функції Ханойських веж ще не викликає помилку переповнення стека `RecursionError`.

Приклад реалізації мовою Python:

```
python
import sys
sys.setrecursionlimit(20000)

def hanoi(n, source, target, auxiliary, moves=None):
    if moves is None: moves = [0]
    if n == 1:
        moves[0] += 1
        return moves[0]
    hanoi(n-1, source, auxiliary, target, moves)
    moves[0] += 1
    hanoi(n-1, auxiliary, target, source, moves)
    return moves[0]

print("Ханойські вежі — кількість переміщень:")
for n in [1, 2, 3, 5, 10, 15, 20]:
    moves = hanoi(n, 'A', 'C', 'B')
    theoretical = 2**n - 1
    status = "OK" if moves == theoretical else "ERR"
    print(f" n={n:2d}: виконано переміщень = {moves:>8}, теоретично 2^n-1 = {theoretical:>8} {status}")

# Звичайна (не хвостова) та псевдо-хвостова рекурсія факторіала
def factorial_recursive(n):
    if n <= 1:
        return 1
    return n * factorial_recursive(n-1) # множення ПІСЛЯ рекурсивного виклику

def factorial_tail(n, accumulator=1):
    if n <= 1:
        return accumulator
    return factorial_tail(n-1, n*accumulator) # рекурсивний виклик - остання дія

assert factorial_recursive(10) == factorial_tail(10) == 3628800
print("\nОбидві реалізації факторіала дають коректний результат: 10! = 3628800")
```

Результат виконання програми:

Ханойські вежі — кількість переміщень:

n= 1: виконано переміщень =	1, теоретично 2^n-1 =	1 OK
n= 2: виконано переміщень =	3, теоретично 2^n-1 =	3 OK
n= 3: виконано переміщень =	7, теоретично 2^n-1 =	7 OK
n= 5: виконано переміщень =	31, теоретично 2^n-1 =	31 OK
n=10: виконано переміщень =	1023, теоретично 2^n-1 =	1023 OK
n=15: виконано переміщень =	32767, теоретично 2^n-1 =	32767 OK
n=20: виконано переміщень =	1048575, теоретично 2^n-1 =	1048575 OK

Обидві реалізації факторіала дають коректний результат: 10! = 3628800

Висновок за результатами: дерево рекурсивних викликів наївного Фібоначчі демонструє яскраво виражений експоненційний ріст: `fib(30)` потребує 2 692 537 викликів функції, тоді як мемоізована версія для того самого `fib(30)`

виконує лише 59 викликів – прискорення у понад 45 000 разів; для `fib(50)` наївна версія практично неспроможна завершитися за розумний час, тоді як мемоїзована обчислює результат лише за 99 викликів. Це наочно ілюструє критичну важливість мемоізації (динамічного програмування) для рекурсивних алгоритмів з деревоподібною структурою, що містять повторювані підзадачі. Задача Ханойських веж підтверджує теоретичну формулу $2^n - 1$ переміщень абсолютно точно для всіх перевірених n – це класичний приклад деревоподібної рекурсії, де кількість викликів (а не лише переміщень) також зростає експоненційно, і жодна мемоізація тут не допоможе, оскільки в цій задачі підзадачі не повторюються (кожен виклик `hanoi(k, ...)` із конкретною комбінацією стрижнів унікальний).

Контрольні питання

1. Дайте визначення рекурсії. Які дві обов'язкові складові повинна мати будь-яка рекурсивна функція?
2. Опишіть механізм роботи стека викликів при виконанні рекурсивної функції.
3. У чому різниця між лінійною та деревоподібною рекурсією? Наведіть приклад кожної.
4. Що таке хвостова рекурсія? Чому в мові Python хвостова форма запису не дає виграшу в пам'яті порівняно зі звичайною рекурсією?
5. Що таке пряма та непряма (взаємна) рекурсія?
6. Поясніть, чому наївний рекурсивний розрахунок чисел Фібоначчі має експоненційну складність $O(2^n)$.
7. Що таке мемоізація і як вона змінює складність рекурсивного алгоритму обчислення чисел Фібоначчі?
8. У яких випадках рекурсивний розв'язок доцільніший за ітеративний, а в яких — навпаки?
9. Чому задача Ханойських веж не може бути прискорена мемоізацією, на відміну від задачі обчислення чисел Фібоначчі?

Зміст звіту

1. Тема, мета роботи.
2. Розгорнуті ручні трасировки стека викликів і дерева рекурсії (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Графік порівняння наївної й мемоїзованої версій Фібоначчі.
5. Висновки.